

PROJECT 3: GUIDE

To create a powerful, flexible, and scalable architecture for hosting your WooCommerce site with advanced microservices, containerization, security, and monitoring. This hybrid solution leverages the strengths of each service:

PART 1: AWS

- **AWS Lightsail** for hosting the core WooCommerce site in a simplified and cost-effective manner.
- **AWS Lambda** for event-driven microservices, reducing the need for always-on infrastructure.
- **Amazon EKS (Elastic Kubernetes Service)** for running containerized microservices that can be easily scaled and managed.

Let's break down how to combine these services and deploy a fully functional e-commerce platform using WooCommerce, Lambda, and EKS.

Overview of the Architecture:

1. **Lightsail** for hosting the main WooCommerce site.
2. **Lambda** for event-driven microservices (e.g., order processing, payment integrations, notifications).
3. **EKS** for managing other complex microservices like inventory management, shipping, and APIs that need to scale.

Service Classification:

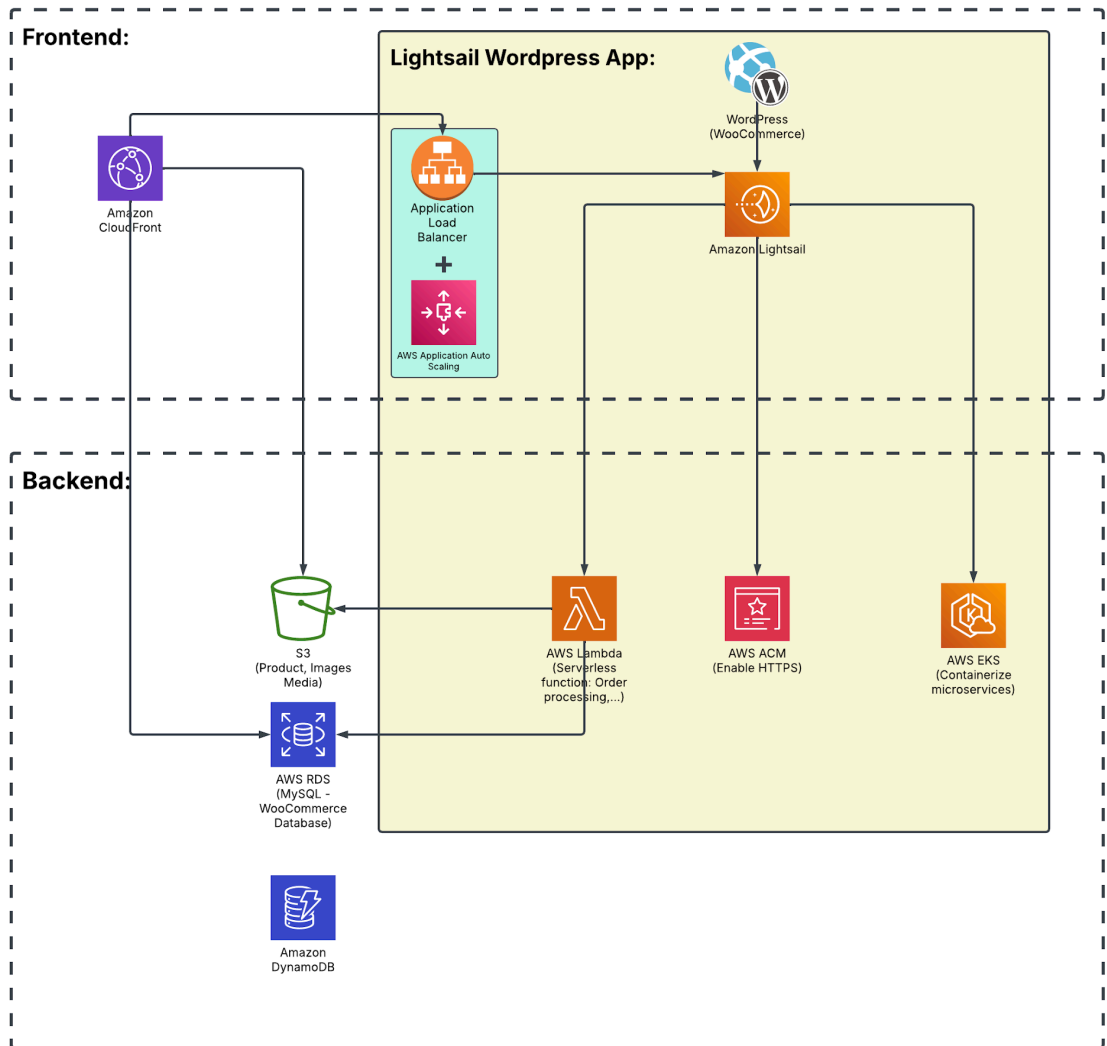
- **Frontend Services:**
 - **Amazon Route 53** (DNS & Domain)
 - **AWS Lightsail** (WooCommerce Website)
 - **Lightsail Load Balancer** (Traffic Handling)
 - **Amazon S3** (Product Images & Media)
 - **CloudFront CDN** (Optional, for Faster Media Delivery)

- **Backend Services:**

- **AWS Lambda** (Order Processing, Image Resizing)
- **Amazon API Gateway** (Microservice Communication)
- **Amazon RDS (MySQL)** (WooCommerce Database)
- **Amazon EKS** (Containerized Microservices)

- **Security & Monitoring Services:**

- **AWS WAF & Shield** (Protection Against Attacks)
- **AWS CloudWatch & GuardDuty** (Monitoring & Logging)
- **Security Groups & VPC** (Network Security)



1. Hosting the WooCommerce Site on AWS Lightsail

Step 1: Create a Lightsail Instance

- **Choose a Blueprint:** Create a **WordPress instance** on Lightsail, which includes WooCommerce out-of-the-box or can be installed from the WordPress dashboard.

- Go to **Lightsail Console > Create Instance**.
- Select the **WordPress Blueprint**.
- Choose a plan (use the free tier or a low-cost plan).
- Choose a region (select the nearest region to your audience).
- Set up an SSH key pair to access the instance.

Step 2: Configure WooCommerce

- Once the Lightsail instance is up, you can access the instance via <http://<your-lightsail-public-ip>>.
- Complete the WordPress setup process, then go to **Plugins > Add New** to install and activate **WooCommerce**.

Step 3: Enable SSL/TLS Encryption

- Use **AWS Certificate Manager (ACM)** to create an SSL certificate.
- Set up a **Lightsail Load Balancer** to provide SSL termination and manage HTTPS traffic.

2. Microservices Using AWS Lambda

Step 1: Use Lambda for Event-Driven Functions

Lambda will handle specific tasks such as:

- **Order Processing:** When an order is placed, Lambda can process it, send email notifications, update inventory, or initiate payment processing.
- **Image Resizing:** When a product image is uploaded to an S3 bucket, Lambda can resize it.
- **Notifications:** Lambda can trigger notifications via **SNS** (Simple Notification Service) for customer orders, abandoned carts, etc.

Example: write Lambda function for Order Processing

Step 2: Trigger Lambda with Events

- Use **Amazon API Gateway** to expose HTTP endpoints that trigger Lambda functions when specific events occur (e.g., order creation, product upload).
- Use **S3 triggers** to invoke Lambda functions when files (e.g., images) are uploaded.
- Use **SNS** to notify Lambda for other asynchronous tasks like low inventory.

3. Running Containerized Microservices with EKS

While Lightsail is great for hosting WordPress, **Amazon EKS** (Elastic Kubernetes Service) is ideal for running microservices that require containerization and scaling, such as:

- Inventory management system.
- API gateway to route traffic between microservices.
- Payment processing services.
- Shipping and tax calculation services.

Step 1: Set Up Amazon EKS Cluster

Create an **EKS cluster** using **eksctl** or the AWS Console.

```
eksctl create cluster --name ecommerce-cluster --region us-west-2  
--nodegroup-name workers --node-type t2.micro --nodes 2
```

Ensure that your **kubectl** is configured to interact with the EKS cluster:

```
aws eks --region us-west-2 update-kubeconfig --name ecommerce-cluster
```

Step 2: Deploy Microservices in EKS

- **Microservice Example:** Let's deploy an **Inventory Management Service** as a Docker container.

First, build your Docker container for the service:

```
FROM node:14  
COPY ./app /usr/src/app  
WORKDIR /usr/src/app  
RUN npm install  
EXPOSE 8080  
CMD ["node", "server.js"]
```

- Push the container to **Amazon ECR** (Elastic Container Registry).

Create a Kubernetes deployment and service YAML file:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: inventory-service
spec:
  replicas: 2
  selector:
    matchLabels:
      app: inventory
  template:
    metadata:
      labels:
        app: inventory
    spec:
      containers:
        - name: inventory-service
          image: <your-ecr-repository-url>/inventory-service:latest
          ports:
            - containerPort: 8080
```

```
apiVersion: v1
kind: Service
metadata:
  name: inventory-service
spec:
  selector:
    app: inventory
  ports:
    - protocol: TCP
      port: 80
      targetPort: 8080
  type: LoadBalancer
```

Step 3: Expose Services in EKS

- Create a **LoadBalancer service** to expose your microservices externally.

- You can use **API Gateway** to route traffic from your WooCommerce site (running on Lightsail) to these microservices via Kubernetes services.

4. Security Best Practices

Step 1: Use IAM Roles and Policies

- Assign appropriate **IAM roles** for Lightsail, Lambda, and EKS nodes, ensuring the least privilege.
- For Lambda, ensure that the functions only have access to the specific resources they need (e.g., S3, SNS).

Step 2: Network Security

- Use **VPC** (Virtual Private Cloud) and configure security groups to restrict access.
- Set up a **private VPC** for your Kubernetes cluster (EKS) and **public subnets** for your Lightsail instance and Load Balancers.

Step 3: Secure Communication

- Ensure **SSL/TLS encryption** is enabled between all services (Lightsail, EKS, Lambda).
- Use **AWS WAF** (Web Application Firewall) for protecting your application from common web exploits.

5. Monitoring and Logging

Step 1: AWS CloudWatch for Lambda and EKS

- Use **CloudWatch Logs** to capture logs from your Lambda functions and EKS pods.
- Set up CloudWatch Alarms to monitor application health and resource usage for both Lambda and EKS.

Step 2: Prometheus and Grafana for Kubernetes

- You can deploy **Prometheus** and **Grafana** on EKS for advanced monitoring and visualization of Kubernetes cluster performance and resource utilization.

Step 3: Lightsail Metrics

- Use **Lightsail Monitoring** to track the performance of your WordPress instance.
- Enable **CloudWatch Metrics** for detailed resource usage insights (e.g., CPU, memory, disk).

6. Auto-Scaling

Step 1: Auto-Scaling for EKS

- Use **Horizontal Pod Autoscaling (HPA)** to automatically scale the number of Kubernetes pods based on CPU or memory usage.
- Ensure that the **EKS cluster** is set to scale with the growing load.

Step 2: Lightsail Auto-Scaling

- For Lightsail, you can manually increase the instance size or use **Lightsail Load Balancer** to distribute traffic to multiple instances.

7. Backup Strategy

- **Lightsail**: Schedule **automatic backups** for your WooCommerce site's instance and database.
 - **EKS**: Use **Amazon EFS** for persistent storage and **RDS** for database backups.
 - Use **AWS Backup** for consistent backups of your resources.
-

Final Combined Architecture

1. **Lightsail** for hosting WooCommerce with a simplified setup.
2. **AWS Lambda** for handling microservices like order processing, payment integration, and image resizing.
3. **EKS** for running complex, containerized microservices that need to scale (inventory management, APIs, etc.).
4. **VPC** to ensure network isolation and security.
5. **AWS WAF** for protecting against common web vulnerabilities.

6. **CloudWatch and Prometheus** for monitoring and alerting.
7. **Auto-scaling** to manage dynamic traffic loads.

This solution is a powerful hybrid that combines **Lightsail's** simplicity for hosting WordPress/WooCommerce, **Lambda's** serverless capabilities for event-driven workflows, and **EKS** for complex, scalable microservices. The architecture will be both cost-effective and scalable while offering flexibility in service management.

Summary of Security Best Practices Applied:

1. **IAM & Least Privilege:** Restrict access across all AWS services.
2. **Encryption:** Use AWS KMS, SSL/TLS, and S3 encryption for data protection.
3. **Network Security:** Use **Security Groups, VPCs, and AWS WAF** to control traffic.
4. **DDoS Protection:** Implement **AWS Shield and rate limiting** to block attacks.
5. **Monitoring & Logging:** Enable **CloudWatch, CloudTrail, and GuardDuty** for security insights.
6. **Backup & Recovery:** Use **Lightsail snapshots, RDS backups, and S3 versioning**.

Reference:

<https://docs.aws.amazon.com/lightsail/latest/userguide/amazon-lightsail-connecting-buckets-to-wordpress.html>

<https://www.youtube.com/watch?v=ZYvPZURkPBM>

<https://docs.aws.amazon.com/whitepapers/latest/best-practices-wordpress/reference-architecture.html>

PART 2: AZURE

1. Using Azure App Service for WooCommerce and WordPress

As previously mentioned, **Azure App Service** is perfect for hosting WordPress and WooCommerce because it provides a fully managed platform with built-in scaling, patching, and security.

- **WordPress and WooCommerce:** Host your WordPress with the WooCommerce plugin in **Azure App Service**. This is ideal for your main e-commerce platform.
- **Scaling:** Azure App Service can automatically scale based on traffic, so you don't have to manage the underlying infrastructure. Use App Service Plans to define the scaling parameters.

2. Integrating Azure Kubernetes Service (AKS) for Other Site Functions

Now, let's look at how to leverage **AKS** for additional site functionality.

Use Case for AKS: Microservices Architecture

You can break your application into smaller microservices and deploy them in **AKS**. Some examples of microservices that would work well with AKS include:

A. Payment Processing Service:

While WooCommerce can handle basic payment integrations, complex or custom payment gateways (e.g., subscriptions, dynamic billing, third-party payment processors) can be hosted in AKS.

Use AKS to manage and scale the payment processing logic separately from your WooCommerce app.

B. Inventory Management Service:

Create a **microservice** that manages inventory updates, stock notifications, and product availability. You can host it in AKS to scale independently of WooCommerce.

C. Search Service (e.g., Elasticsearch):

If you need advanced search functionality (e.g., faster product searches, personalized recommendations), you can deploy **Elasticsearch** on AKS for high

availability and scalability.

D. **Email Notifications and Messaging:**

For complex email workflows (order confirmations, marketing emails, etc.), you can deploy an **email processing microservice** on AKS to handle asynchronous email sending.

E. **Background Processing:**

Heavy background tasks (like generating reports, processing data, or handling long-running jobs) can be offloaded to AKS, ensuring that these tasks don't affect the performance of your main WooCommerce app.

3. **Database Management with Azure Database for MySQL**

Since you're using **WooCommerce**, you'll need a **MySQL** database to manage all of your e-commerce data. You can use **Azure Database for MySQL** for this purpose.

- **Database Connections:** Your WooCommerce App Service and the microservices in AKS can both interact with this **Azure Database for MySQL**.
 - **Security:** Ensure that **database credentials** and sensitive information are stored securely in **Azure Key Vault**.
-

4. **Using Azure Functions for Event-Driven Tasks**

You can still leverage **Azure Functions** to handle lightweight or event-driven tasks that are triggered by specific events.

- **Background Jobs:** Use **Azure Functions** for background tasks such as sending emails, processing order confirmations, or integrating with external systems.
 - **Blob Storage Events:** If you're using **Blob Storage** for storing images (e.g., product images), you can trigger **Azure Functions** to resize images or process files when they are uploaded to the storage account.
-

5. Secure the Solution with Azure Key Vault

To securely manage secrets, certificates, and other sensitive data:

- **Azure Key Vault:** Use **Key Vault** to securely store database credentials, API keys, and SSL certificates for both your **App Service** and **AKS**-hosted services.
 - **App Service Integration:** **Azure App Service** can integrate with **Key Vault** to securely retrieve secrets without hardcoding them in the application code.
 - **AKS Integration:** Similarly, **AKS** can access secrets stored in **Azure Key Vault** via **managed identities** to avoid hardcoded credentials in your microservices.
-

6. Scaling and Monitoring

You can scale both **Azure App Service** and **AKS** independently:

- **Azure App Service:** Use **Auto-Scaling** for WooCommerce on App Service, which will scale up or down based on traffic and load.
 - **Azure Kubernetes Service (AKS):** For microservices, you can scale the pods based on demand. **Horizontal Pod Autoscaling** in AKS will automatically increase or decrease the number of running pods based on traffic and resource utilization.
-

7. Monitoring and Logging:

- **Azure Monitor:** For both **App Service** and **AKS**, you can use **Azure Monitor** to collect telemetry data, logs, and metrics. You can configure alerts for performance issues, resource consumption, and failures.
 - **Application Insights:** Use **Application Insights** for detailed performance monitoring, error tracking, and debugging for your **WooCommerce App Service** and **AKS microservices**.
 - **Azure Log Analytics:** Aggregate and query logs from **AKS**, **App Service**, and other components into **Azure Log Analytics** to troubleshoot issues across the entire system.
-

8. CI/CD Pipeline

To deploy your **WooCommerce app** and **microservices**:

- Use **Azure DevOps** or **GitHub Actions** to set up **CI/CD pipelines** for both **App Service** and **AKS**.
 - **App Service**: Automate deployment of WordPress and WooCommerce.
 - **AKS**: Automate the deployment of microservices, ensuring that the services in AKS are always up-to-date with your latest changes.
-

9. Summary of Solution

Here's an updated architecture combining **Azure App Service** for WooCommerce and **Azure Kubernetes Service (AKS)** for additional functions:

1. **Azure App Service**: Hosts WooCommerce and WordPress.
2. **Azure Kubernetes Service (AKS)**: Hosts microservices such as payment processing, inventory management, search services (Elasticsearch), and background processing.
3. **Azure Functions**: Handles event-driven tasks such as image processing or sending notifications.
4. **Azure Database for MySQL**: Manages the WooCommerce site's database.
5. **Azure Key Vault**: Securely stores API keys, credentials, and SSL certificates.
6. **Azure Monitor & Application Insights**: For monitoring and diagnostics.

References:

<https://learn.microsoft.com/en-us/azure/architecture/web-apps/>

<https://learn.microsoft.com/en-us/azure/architecture/aws-professional/#web-applications>